

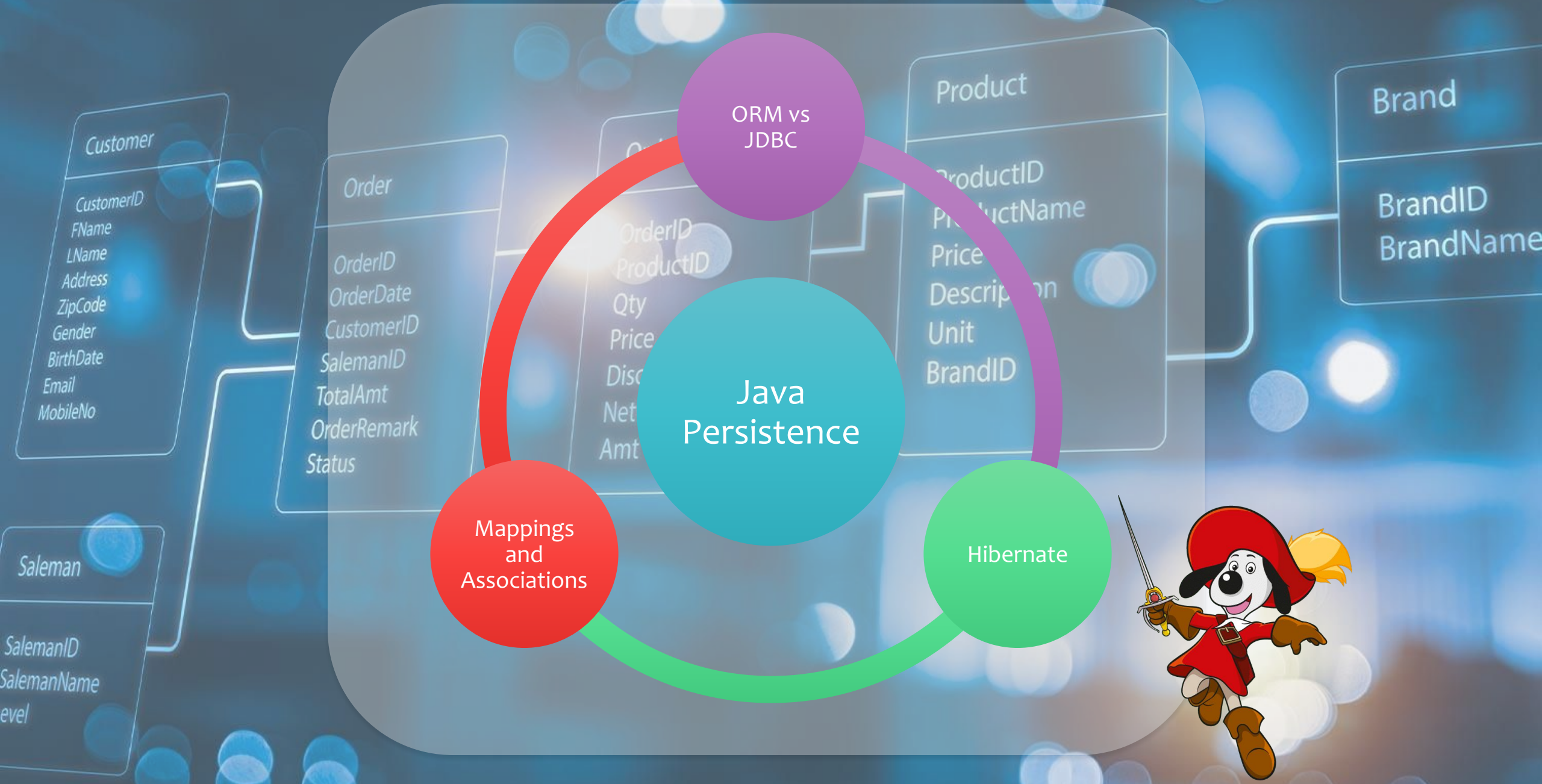
Summarizer

- 21 - 03 - 2023
- Ricardo Ortins



ACADEMIA DE CÓDIGO

To do List



JDBC (Java
Database
Connectivity)

Manage Database

ORM
(Object-Relational
Mapping)



Customer

CustomerID
FName
LName
Address
ZipCode
Gender
BirthDate
Email
MobileNo

Product

Product
Product

Brand

BrandID
BrandName

Saleman

SalemanID
SalemanName
Level

Pros and Cons JDBC

Pros of using JDBC:

- Flexibility
- High performance
- Easy to learn
- Support for complex queries

Cons of using JDBC:

- Tedious and error-prone
- Time-consuming
- Database-specific



Customer

CustomerID
FName
LName
Address
ZipCode
Gender
BirthDate
Email
MobileNo

Order

OrderID
ProductID
Qty
Price
Discount
NetPrice
Amt

SalemanID
TotalAmt
OrderDate
Status

Product

ProductID
ProductName
Price
Description
Unit
BrandID

Brand

BrandID
BrandName

Saleman

SalemanID
SalemanName
Level

Pros and Cons ORM

Pros of using ORM:

- Object-oriented approach
- Simplified database access
- Increased productivity

Cons of using ORM:

- Limited control
- Steep learning curve
- Difficult to do Complex queries



Customer	
CustomerID	
FName	
LName	
Address	
ZipCode	
Gender	
BirthDate	
Email	
MobileNo	

Saleman	
SalemanID	
SalemanName	
Level	

Order	
OrderID	
ProductID	
Qty	
Price	
Discount	
NetPrice	
Amt	

Product	
ProductID	
ProductName	
Price	
Description	
Unit	
BrandID	

Brand	
BrandID	
BrandName	

ORM or JDBC?

Should I choose ORM or JDBC?

It depends on the project requirements.

ORM:

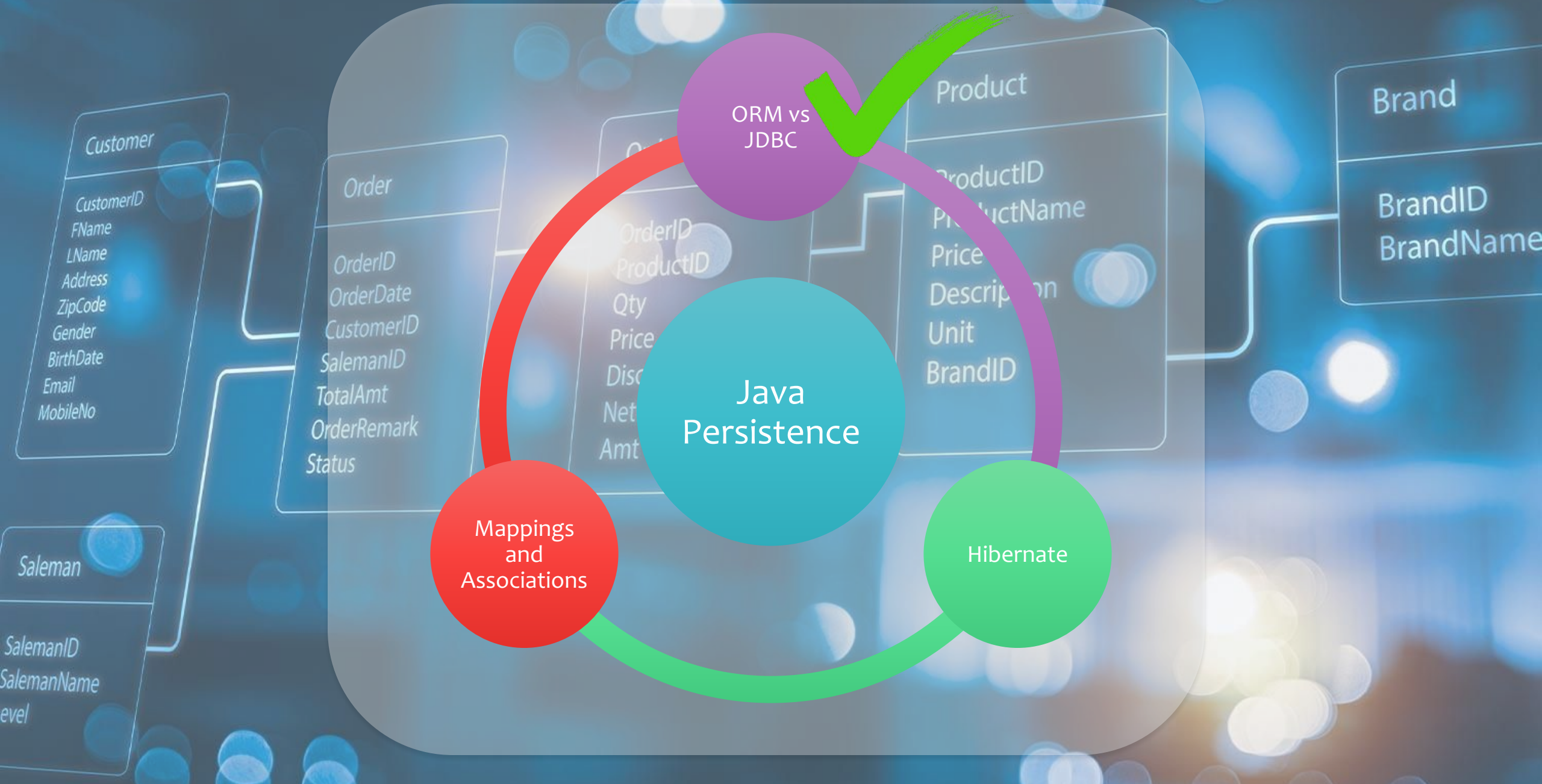
- Simplify database access
- Increase productivity

JDBC:

- Greater control of databases
- Flexibility on databases



To do List



ORM: Hibernate

Key features of Hibernate:

- High-level abstraction layer
- Reduces boilerplate code
- Improve performance
- Reduce time spent on database-related tasks



IMPOSSIBRU

IMPOSSIBRU

JPA: Entity Classes

Key points Entity Classes in JPA:

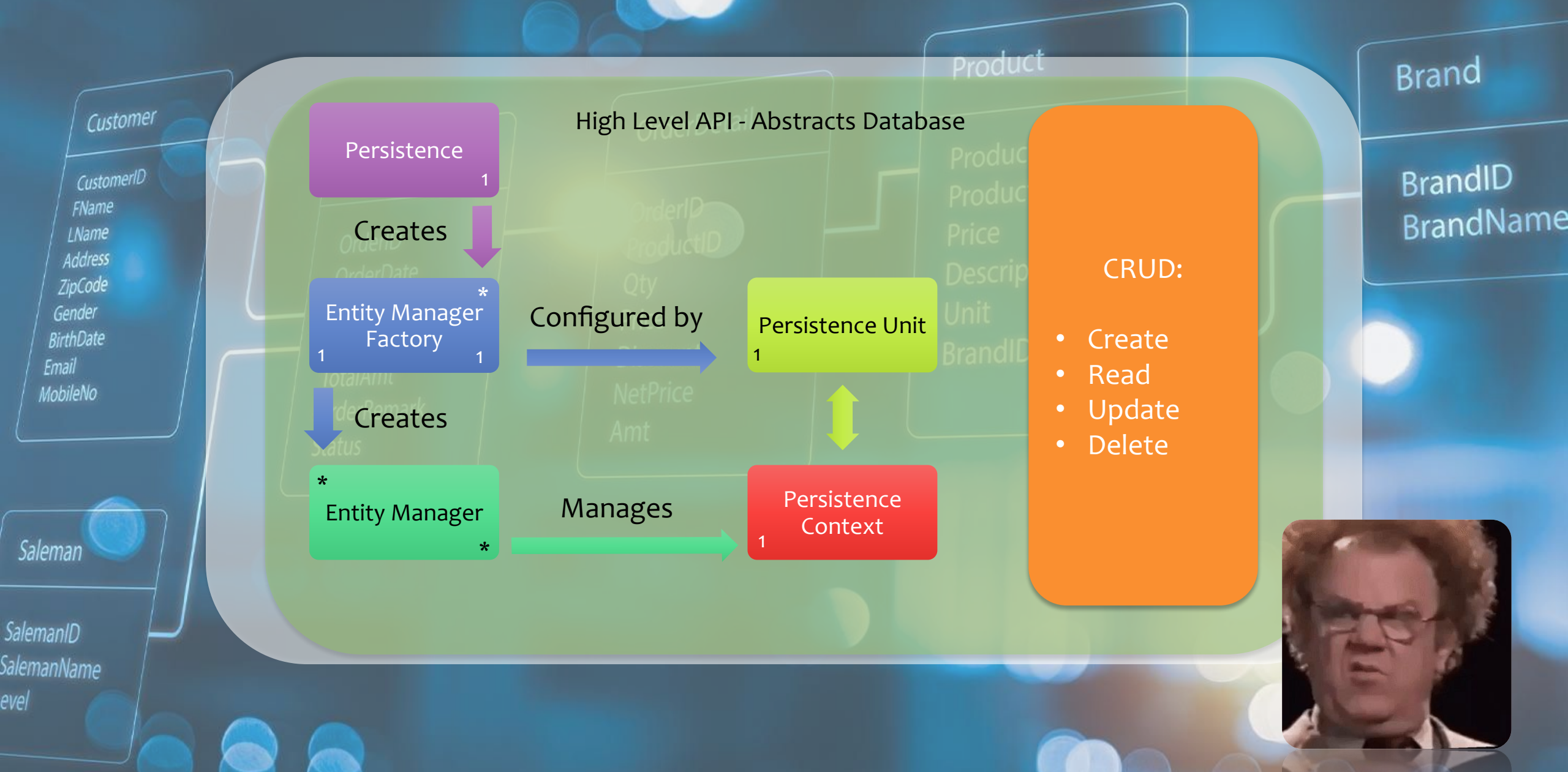
- Must have a no-argument constructor
- Should not be declared final
- Should have an id property to be used as the primary key
- Should have setters and getters for all persistent fields

```
@Entity
public class Employee {
    @Id
    private id;
    private String name;
    private int age;
    private double salary;

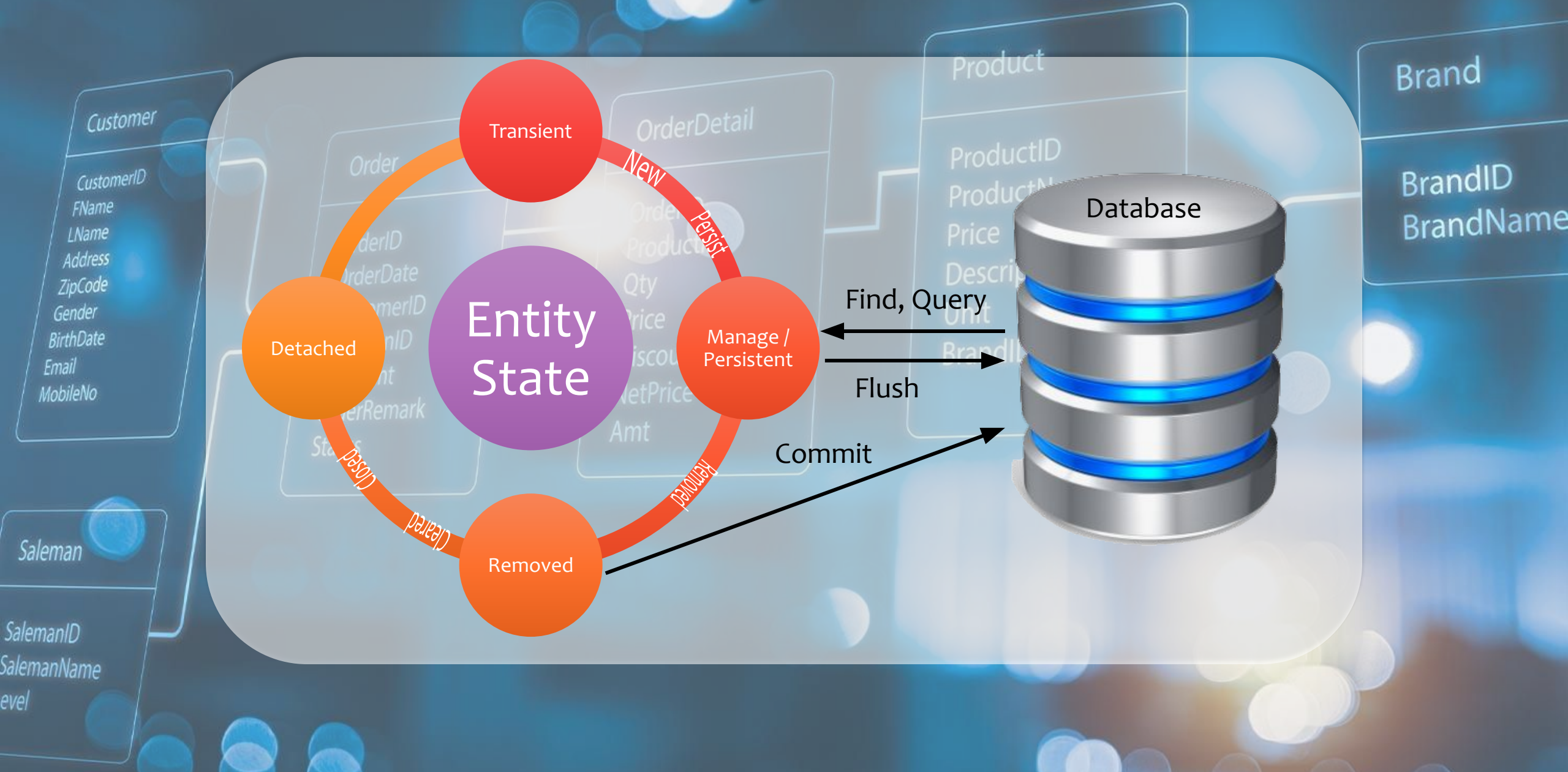
    // getters and setters
}
```

```
// getters and setters
```

JPA Architecture



Entity State



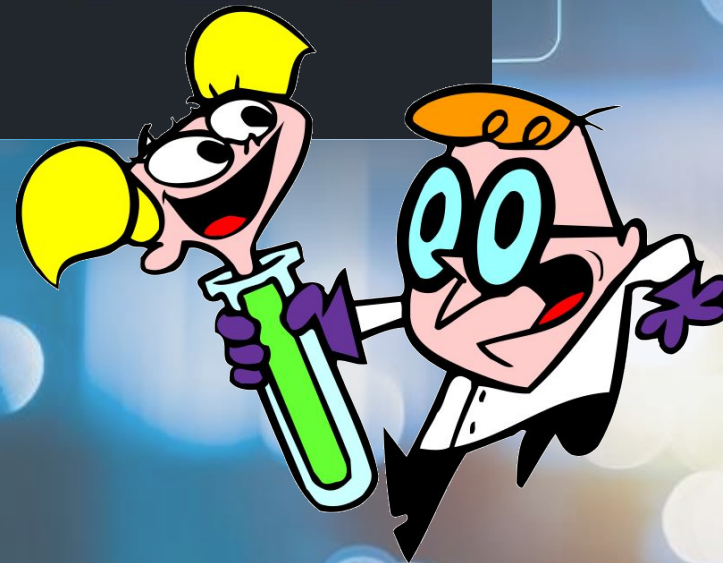
Transient



```
Person person = new Person();
person.setName("John");
person.setAge(30);

EntityManager em = entityManagerFactory.createEntityManager();
em.getTransaction().begin();
em.persist(person);
em.getTransaction().commit();
```

```
em.getTransaction().commit();
em.persist(person);
em.getTransaction().commit();
```



Persistent

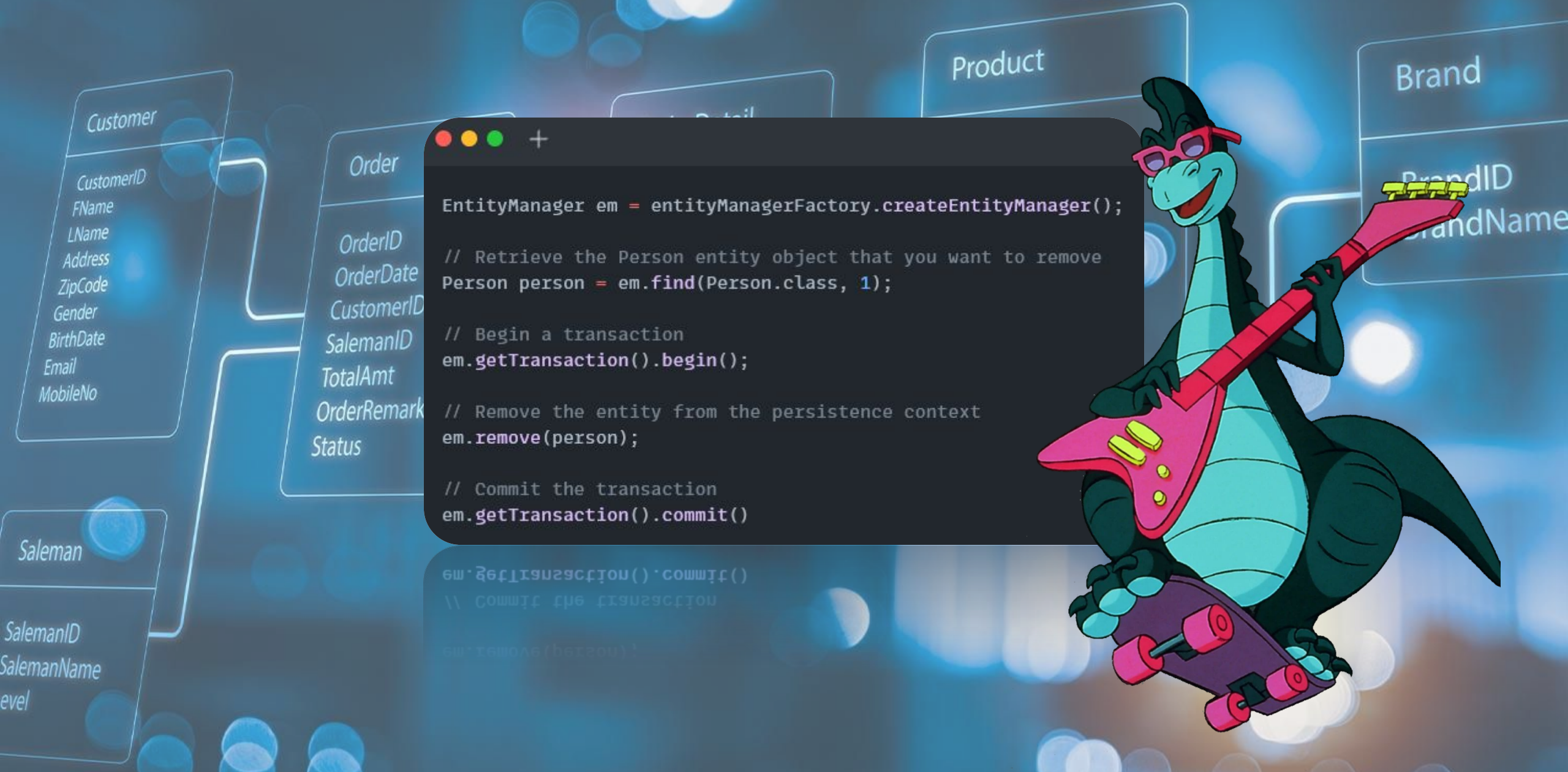


```
EntityManager em = entityManagerFactory.createEntityManager();
Person person = em.find(Person.class, 1);
```

COW And CHICKEN



Removed



Detached

Customer

CustomerID
FName
LName
Address
ZipCode
Gender
BirthDate
Email
MobileNo

Product

Brand

BrandID
BrandName

```
// Assume we have a managed Person entity with ID = 1
EntityManager em = entityManagerFactory.createEntityManager();
em.getTransaction().begin();
Person person = em.find(Person.class, 1);
em.getTransaction().commit();

// Now the person entity is detached
person.setName("John Smith");

// We can still modify the detached entity, but changes will not be persisted
// unless we explicitly re-attach the entity to the persistence context
em.getTransaction().begin();
em.merge(person);
em.getTransaction().commit();
```

```
em.getTransaction().commit();
em.merge(person);
em.getTransaction().commit();
```

// merge will explicitly re-attach the entity to the persistence context



Saleman

SalemanID
SalemanName
Level

To do List

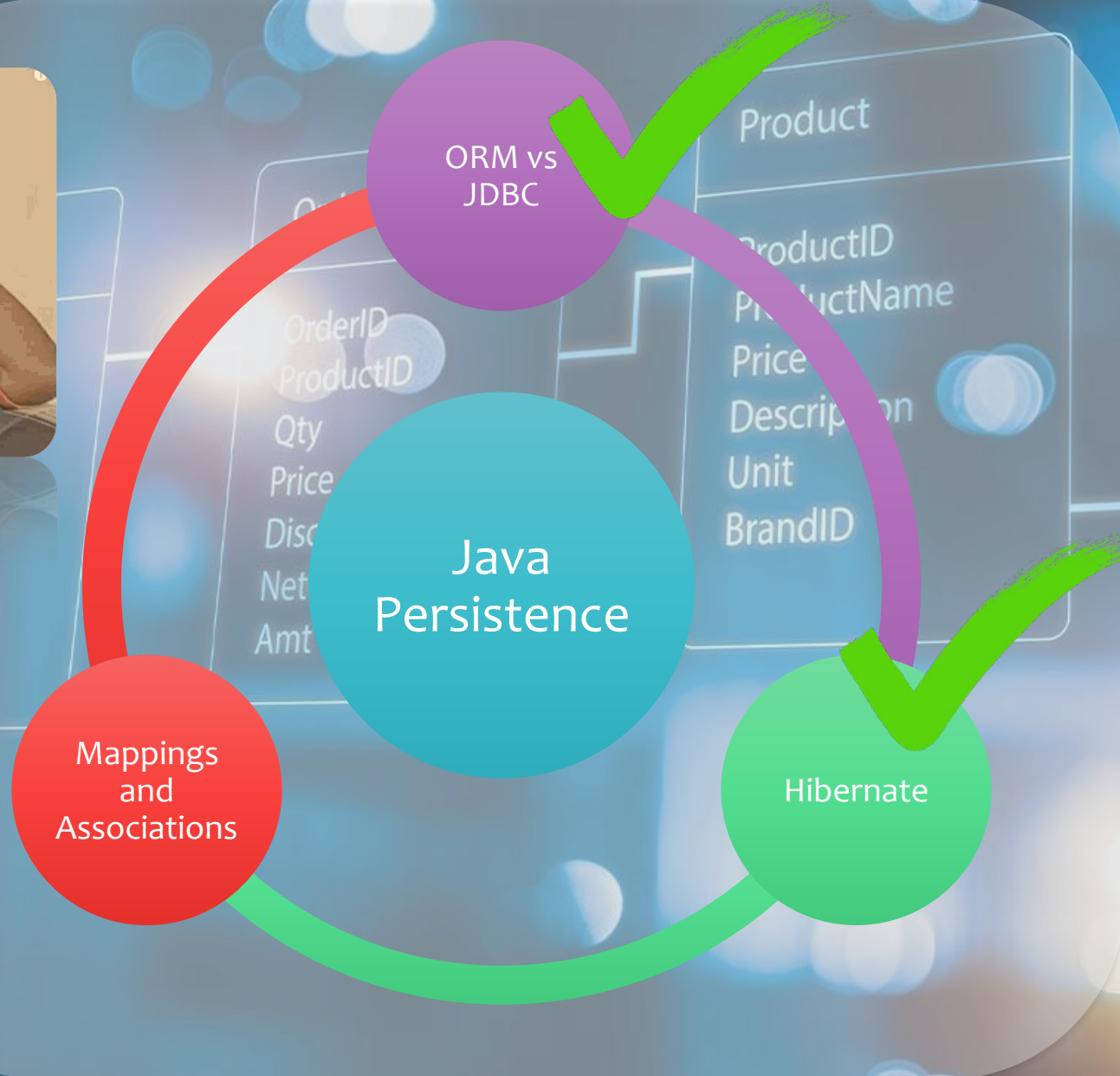


Gender
BirthDate
Email
MobileNo

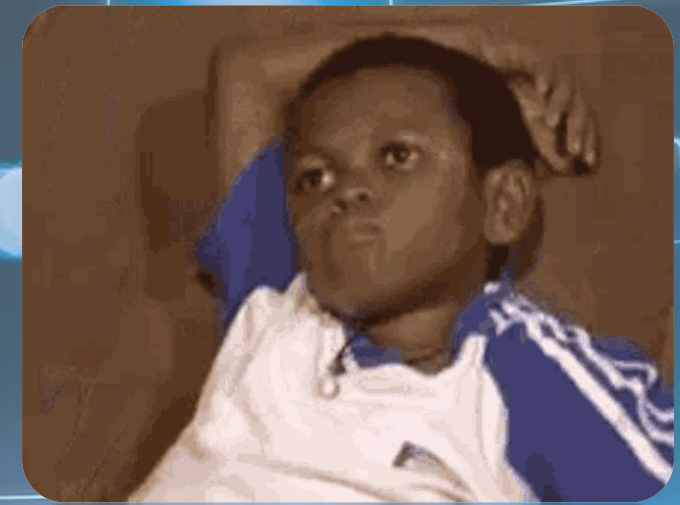
CustomerID
SalemanID
TotalAmt
OrderRemark
Status

Saleman

SalemanID
SalemanName
Level



Identifier Generation



AUTO

IDENTITY

SEQUENCE

TABLE

Locking and TimeStamps

```
@Entity
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Version
    private Integer version;

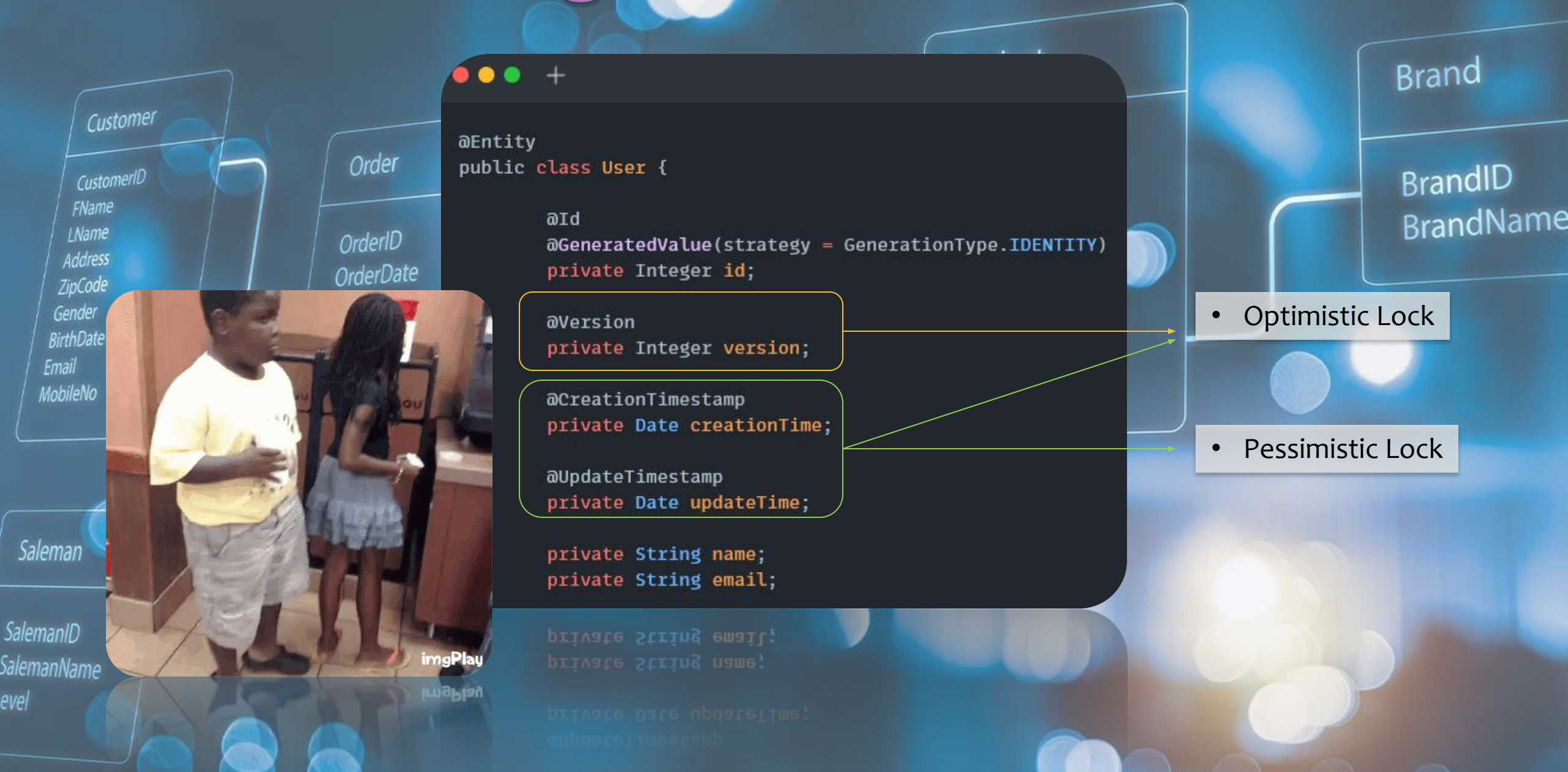
    @CreationTimestamp
    private Date createTime;

    @UpdateTimestamp
    private Date updateTime;

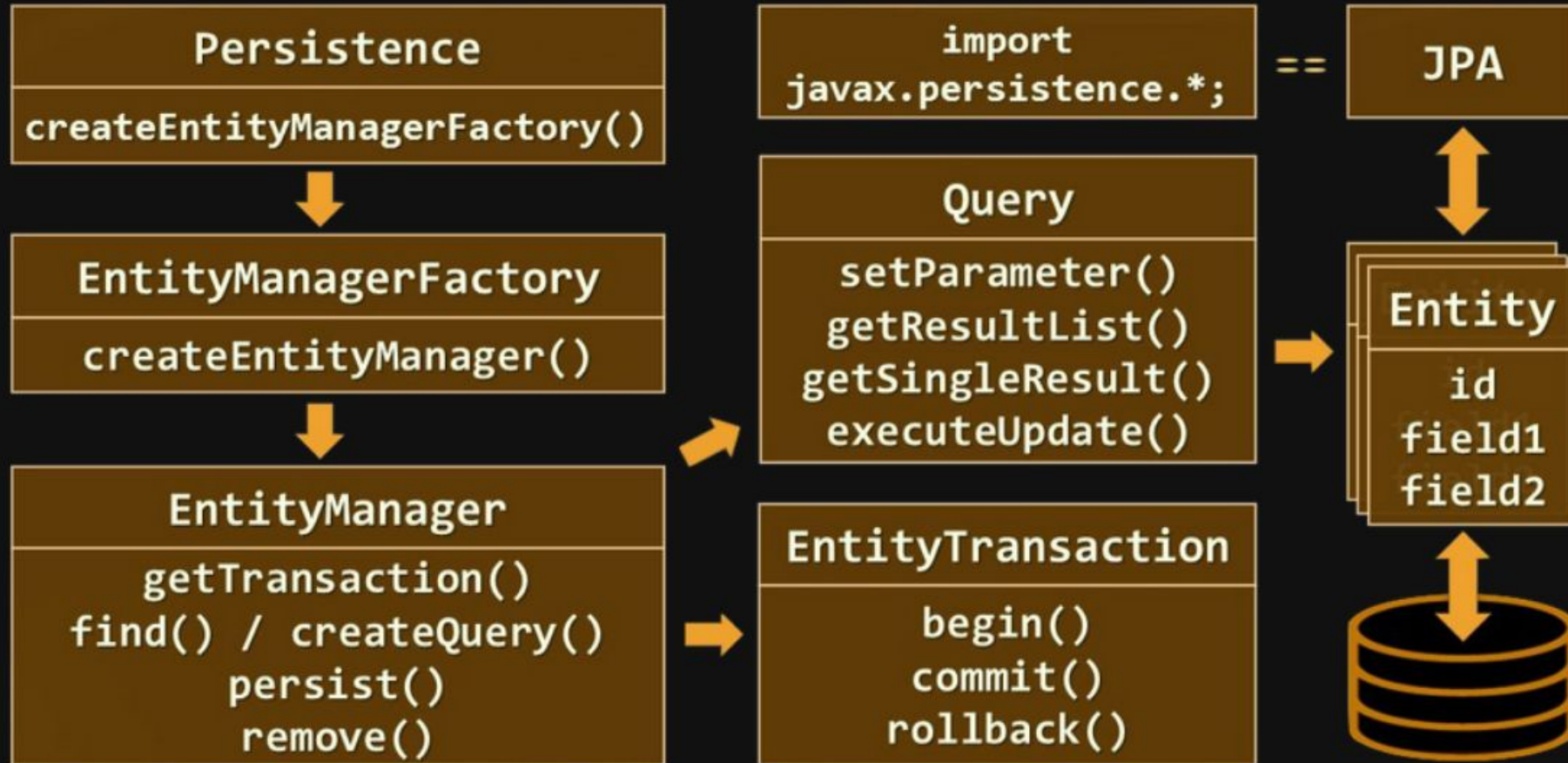
    private String name;
    private String email;
```

- Optimistic Lock

- Pessimistic Lock



JPA API Overview



JPA API classes are found on the `javax.persistence.*` package

Inheritance Mapping

Mapped Superclass

Table per Class

Single Table

Joined Table



Mapped Superclass



```
@MappedSuperclass
public class Person {
    @Id
    @GeneratedValue
    private Integer id;

    private String firstName;

    private String lastName;

    // Getters and setters
}

@Entity
@Table(name = "students")
public class Student extends Person {
    private String major;

    // Getters and setters
}

@Entity
@Table(name = "instructors")
public class Instructor extends Person {
    private String department;

    // Getters and setters
}
```

OrderDetail

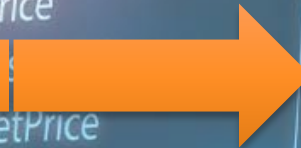
OrderID
ProductID

Qty

Price

Dis
NetPrice

Amt



Product

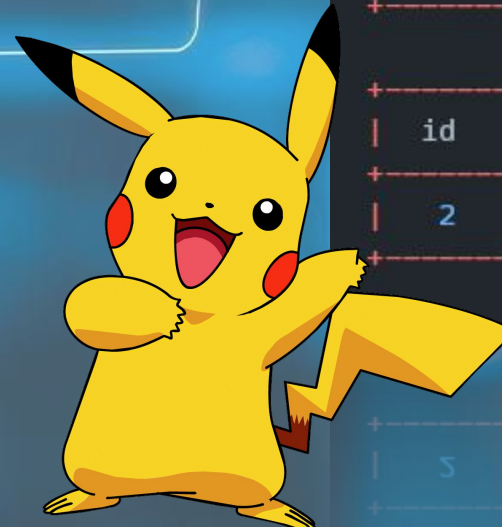
id	firstName	lastName
1	Alice	Smith
2	Bob	Jones

id	major
1	CS

id	department
2	Math

Brand

BrandID
BrandName

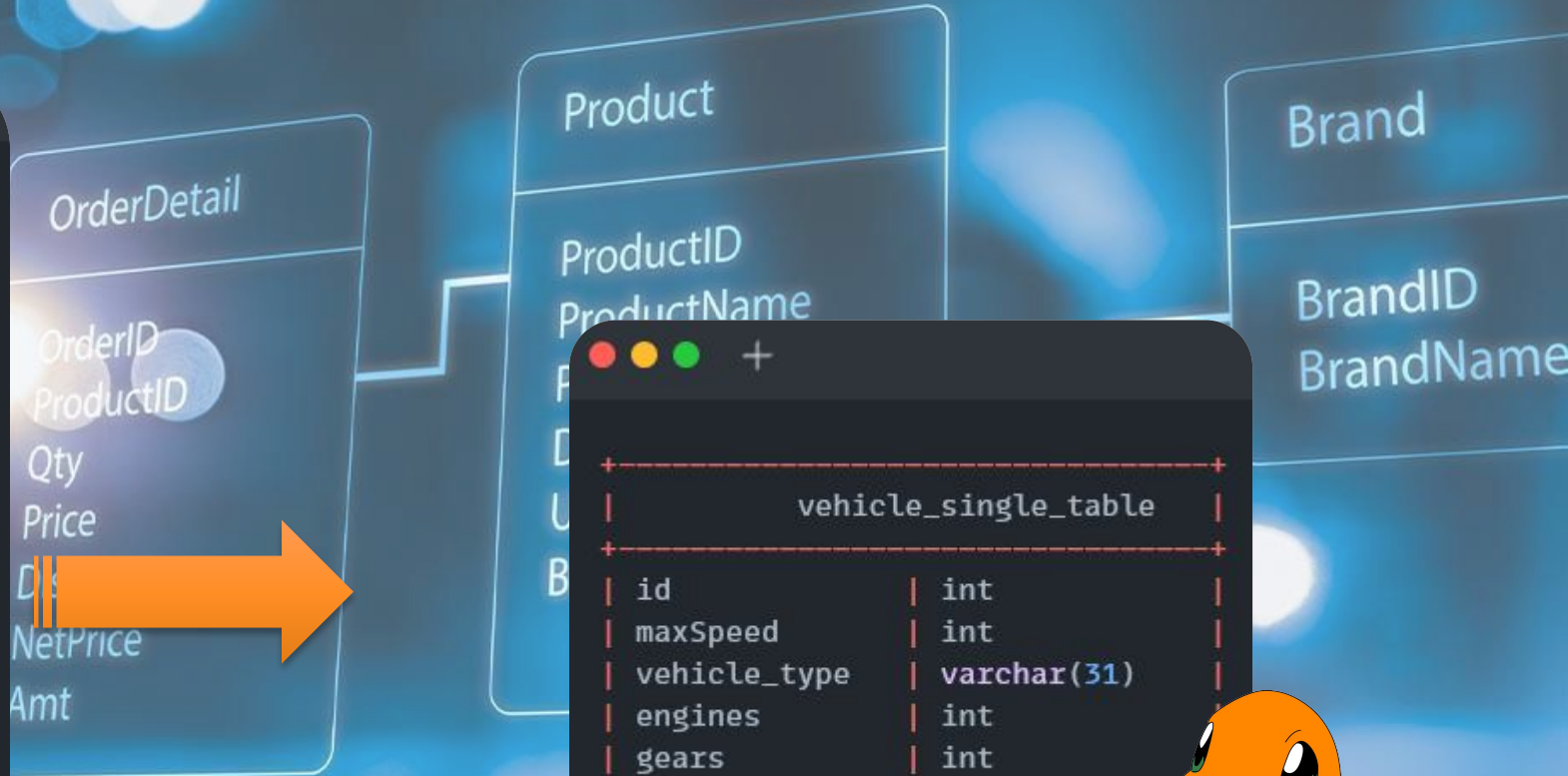


Single Table Inheritance

```
@Entity(name="BoatSingleTable")
@DiscriminatorValue("boat")
public class Boat extends Vehicle {
    private Integer engines;
}

@Entity(name="CarSingleTable")
@DiscriminatorValue("car")
public class Car extends Vehicle {
    private Integer gears;
}

@Entity(name="VehicleSingleTable")
@Table(name = "vehicle_single_table")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(
    name = "vehicle_type",
    discriminatorType = DiscriminatorType.STRING
)
public abstract class Vehicle {
    @Id
    private Integer id;
    private Integer maxSpeed;
}
```



vehicle_single_table	
id	int
maxSpeed	int
vehicle_type	varchar(31)
engines	int
gears	int

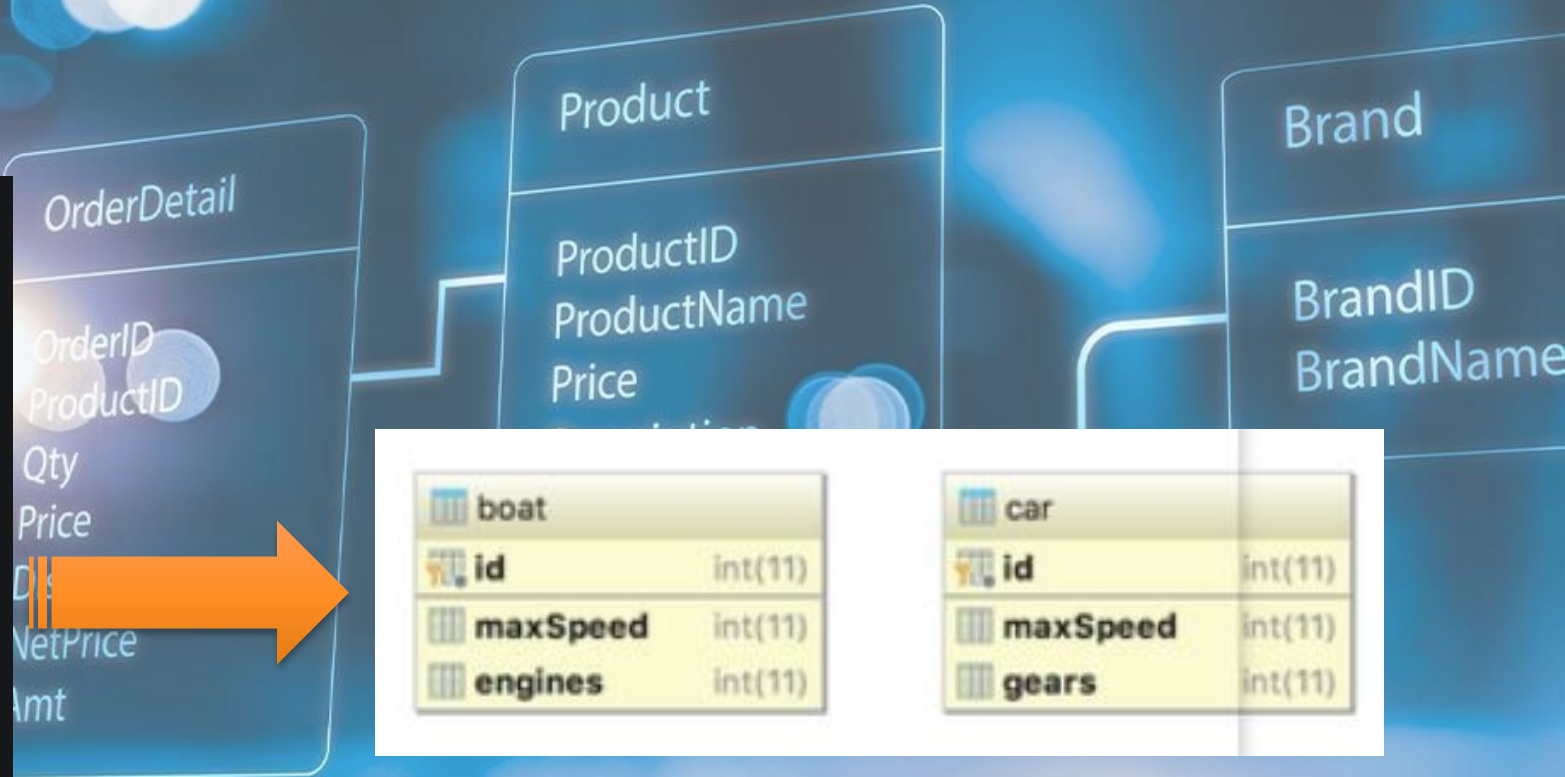


Table Per Class Inheritance

```
@Entity
@Table(name = "boat_table_per_class")
public class Boat extends Vehicle {
    private Integer engines;
}

@Entity
@Table(name = "car_table_per_class")
public class Car extends Vehicle {
    private Integer gears;
}

@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class Vehicle {
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE)
    private Integer id;
    private Integer maxSpeed;
}
```

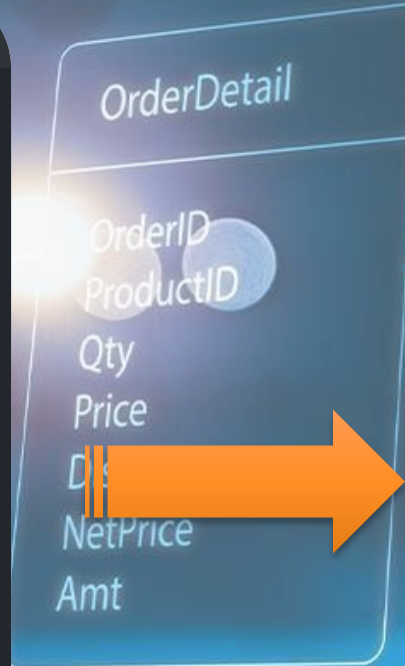


Joined Table Inheritance

```
@Entity(name = "BoatJoinedTable")
@Table(name = "boat_joined_table")
public class Boat extends Vehicle {
    private Integer engines;
}

@Entity(name = "CarJoinedTable")
@Table(name = "car_joined_table")
public class Car extends Vehicle {
    private Integer gears;
}

@Entity(name = "VehicleJoinedTable")
@Table(name = "vehicle_joined_table")
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Vehicle {
    @Id
    private Integer id;
    private Integer maxSpeed;
}
```



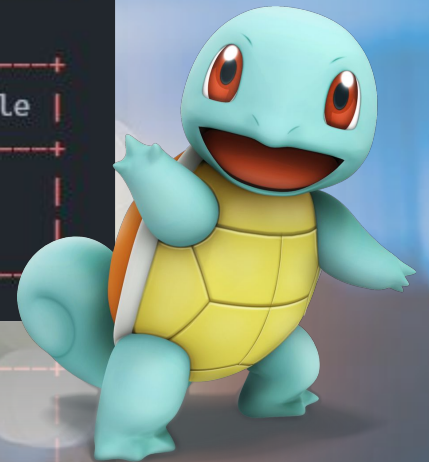
```
+-----+
|      vehicle_joined_table      |
+-----+
| id          | int  |
| maxSpeed    | int  |
+-----+

+-----+
|      boat_joined_table        |
+-----+
| id          | int  |
| engines     | int  |
+-----+

+-----+
|      car_joined_table         |
+-----+
| id          | int  |
| gears       | int  |
+-----+
```

Brand

BrandID
BrandName



That's all I

